TALLER · SESIÓN 10

El login con backend en Python, SQLite y Google

Paso a paso: el proyecto login_screen completo. Registro, inicio de sesión con correo y con Google, página privada, una API de datos y la prueba en el emulador y en el celular.

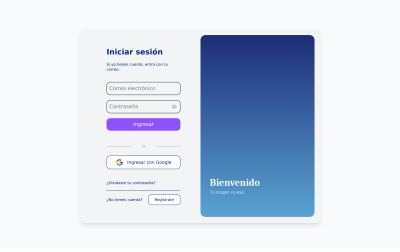
Asignatura	Diseño de Aplicaciones para Dispositivos Móviles	**Programa**	Ingeniería de Software
Docente	Ing. Kevin Torrez	**Duración**	55 minutos en clase; lo que falte, en casa

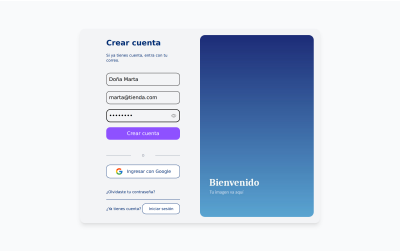
En la sesión 9 hiciste la pantalla de login con Tailwind. Hoy le pones todo lo que le falta por detrás: un **backend** en Python que recibe los datos, una **base de datos** SQLite donde viven los usuarios, una **sesión** que recuerda quién entró, el **inicio de sesión con Google** y una página privada. Y lo pruebas donde se prueba una página móvil: en el emulador y en tu celular.

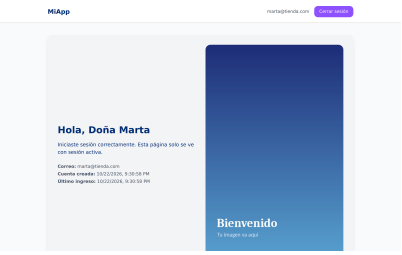


El viaje de un login: el navegador envía, Flask decide, SQLite guarda.

¿Qué es? Flask es una librería de Python para hacer servidores web. **SQLite** es una base de datos que vive en un solo archivo (database.db) y ya viene con Python. **Google Sign-In** deja que la persona entre con su cuenta de Google: Google confirma quién es y tu backend solo verifica esa confirmación.







Iniciar sesión *Crear cuenta (mismo formulario)* *Inicio: solo con sesión*

Antes de empezar

- La carpeta **login_screen** de la sesión 9, con index.html y tu imagen. Si prefieres, el ZIP del aula virtual trae el proyecto completo y terminado.
- VS Code y Chrome o Edge. Internet para instalar Python y las librerías.
- Para Google: una cuenta de Google y acceso a console.cloud.google.com. Si no es posible en clase, ese paso queda para la casa.
- Para la parte final: Android Studio con el emulador, y el celular en la misma Wi-Fi del computador.

La regla del taller: en cada paso, copia el código, ejecútalo y comprueba «lo que debes ver» antes de seguir. Si algo falla, la tabla de problemas comunes está al final.

Paso 1: Python, el entorno y las librerías (8 min)

1. Instala Python 3 desde **python.org/downloads** marcando **Add python.exe to PATH**. Abre una terminal nueva: **python --version** debe responder «Python 3.x».
2. En VS Code: **File → Open Folder → login_screen**. Abre la terminal con **Ctrl+ñ**.
3. Crea la carpeta **backend** y dentro el archivo **requirements.txt** con estas tres líneas:

backend/requirements.txt

```
Flask>=3.0
google-auth>=2.0
requests>=2.0
```

4. En la terminal, uno por uno:

Terminal (PowerShell), en la carpeta login_screen

```
python -m venv .venv
.venv\Scripts\activate
pip install -r backend\requirements.txt
```

Lo que debes ver: la línea empieza con **(.venv)** y pip termina con «Successfully installed Flask… google-auth… requests…».

¿Qué es el entorno virtual? Una carpeta (.venv) donde se instalan las librerías solo para este proyecto. Si PowerShell bloquea el activate, es el arreglo de la sesión 4: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`. Si pip no llega a internet por la red del salón, usa el hotspot del celular.

```
TERMINAL · PowerShell · login_screen

PS C:\Users\Innovacion01\Desktop\login_screen> python --version
Python 3.13.7

PS C:\Users\Innovacion01\Desktop\login_screen> python -m venv .venv
PS C:\Users\Innovacion01\Desktop\login_screen> .venv\Scripts\activate
(.venv) PS C:\Users\Innovacion01\Desktop\login_screen> pip install flask
Collecting flask ... Successfully installed flask-3.1.2 ...

(.venv) PS C:\Users\Innovacion01\Desktop\login_screen> python app.py
 * Serving Flask app 'app'
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.20:5000
Press CTRL+C to quit
```

Así se ve la terminal con los pasos 1 y 3.

Paso 2: La estructura y la base de datos (5 min)

Así queda la carpeta al final. Cada archivo tiene una sola responsabilidad: el frontend no sabe de SQL y el backend no sabe de Tailwind; solo se hablan por la API.

Estructura de login_screen

```
login_screen/
    index.html        el login (y el registro: es el mismo formulario)
    app.js            envía los datos al backend con fetch
    home.html         la página privada
    mobile.js         pinta la base de conocimiento en el home
    login.jpg         tu imagen
    .venv/            el entorno virtual (no se entrega)
    backend/
        app.py          el servidor: rutas, API, sesión y Google
        db.py           la conexión a SQLite
        mobile.py       la API /api/mobile/* (base de conocimiento)
        mobile_data.py  el contenido que se carga en la base de datos
        config.py       el Client ID de Google
        requirements.txt
        database.db     se crea sola al arrancar
```

1. Crea **backend/db.py**. Es pequeño: abre la conexión una vez por petición y la cierra al terminar.

backend/db.py

```
"""Conexión SQLite compartida por la app y los blueprints."""
import sqlite3
from datetime import datetime
from pathlib import Path

from flask import g

BASE_DIR = Path(__file__).resolve().parent
DB_PATH = BASE_DIR / "database.db"


def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
        g.db.row_factory = sqlite3.Row
```

```
    return g.db


def close_db(_exc=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()


def now():
    return datetime.now().isoformat(timespec="seconds")
```

Qué hace

- **DB_PATH** apunta a backend/database.db, al lado de este archivo, sin importar desde dónde se ejecute.
- **get_db()** guarda la conexión en **g**, un espacio de Flask que vive lo que dura una petición; así app.py y mobile.py comparten la misma conexión.
- **row_factory = sqlite3.Row** deja leer las filas por nombre: row["email"].
- **now()** devuelve la fecha y hora como texto, para created_at y last_login.

2. Crea **backend/config.py**. Por ahora vacío; en el paso 6 pegarás aquí el Client ID de Google.

backend/config.py

```
# Configuración del backend
#
# Pega aquí tu "ID de cliente" de Google (termina en .apps.googleusercontent.com).
# Cómo obtenerlo: ver README.md, sección "Login con Google".
GOOGLE_CLIENT_ID = ""
```

Paso 3: El backend, app.py por bloques (12 min)

Crea **backend/app.py** y pega los bloques en orden. Al terminar es un solo archivo. Lee la explicación de cada bloque: es lo que vale en la bitácora.

Bloque 1: importaciones y configuración

backend/app.py (1 de 6)

```
"""
Backend del login (Flask + SQLite).

Ejecutar:
    pip install -r requirements.txt
    python app.py
Luego abrir http://localhost:5000
"""
import os
import re
import sqlite3
from functools import wraps

from flask import Flask, jsonify, redirect, request, send_from_directory, session
from werkzeug.security import check_password_hash, generate_password_hash

from db import BASE_DIR, DB_PATH, close_db, get_db, now
from mobile import bp as mobile_bp, seed_mobile_data

try:
    import config  # backend/config.py
except ImportError:  # pragma: no cover
    config = None

FRONTEND_DIR = BASE_DIR.parent          # carpeta login_screen (index.html, imágenes)
# Solo se sirven estos tipos de archivo del frontend (nunca .db, .py, .venv, ...)
STATIC_EXTENSIONS = {".html", ".js", ".css", ".jpg", ".jpeg", ".png", ".svg", ".webp", ".ico"}

GOOGLE_CLIENT_ID = os.environ.get("GOOGLE_CLIENT_ID") or getattr(config, "GOOGLE_CLIENT_ID", "")

EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")

app = Flask(__name__, static_folder=None)
app.config.update(
    SECRET_KEY=os.environ.get("SECRET_KEY", "cambia-esta-clave-en-produccion"),
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
)


app.teardown_appcontext(close_db)
app.register_blueprint(mobile_bp)
```

- **FRONTEND_DIR** es la carpeta login_screen: desde ahí Flask sirve index.html, app.js y las imágenes.
- **STATIC_EXTENSIONS** es una lista blanca: solo se pueden pedir por la web archivos con estas extensiones. Nadie puede descargar database.db ni app.py.
- **SECRET_KEY** firma la cookie de sesión. **HTTPONLY**: el JavaScript no puede leerla. **SAMESITE**: otras páginas no pueden usarla.
- La API de conocimiento (mobile_bp) se registra como un blueprint: un grupo de rutas que vive en otro archivo.

Bloque 2: la base de datos

backend/app.py (2 de 6)

```
# ------------------------------------------------------------- base de datos
def init_db():
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS users (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                name          TEXT,
                email         TEXT NOT NULL UNIQUE,
                password_hash TEXT NOT NULL,
                created_at    TEXT NOT NULL,
                last_login    TEXT,
                google_id     TEXT UNIQUE
            )
            """
        )
        # migración para bases creadas con la versión anterior
        cols = [r[1] for r in conn.execute("PRAGMA table_info(users)")]
        if "google_id" not in cols:
            conn.execute("ALTER TABLE users ADD COLUMN google_id TEXT")
        # base de conocimiento de apps móviles (/api/mobile/*)
        seed_mobile_data(conn)
```

- **CREATE TABLE IF NOT EXISTS**: crea la tabla la primera vez y no toca nada las siguientes.
- **email UNIQUE**: no puede haber dos usuarios con el mismo correo. **google_id UNIQUE**: une la cuenta con Google.
- La **migración**: si la base se creó con una versión vieja sin google_id, ALTER TABLE agrega la columna sin borrar los usuarios.

Bloque 3: ayudantes

backend/app.py (3 de 6)

```
def start_session(db, user_id):
    db.execute("UPDATE users SET last_login = ? WHERE id = ?", (now(), user_id))
    db.commit()
    session.clear()
    session["user_id"] = user_id
    return db.execute("SELECT * FROM users WHERE id = ?", (user_id,)).fetchone()


def user_to_dict(row):
    return {
        "id": row["id"],
        "name": row["name"],
        "email": row["email"],
        "created_at": row["created_at"],
        "last_login": row["last_login"],
    }


def login_required(view):
    @wraps(view)
    def wrapper(*args, **kwargs):
        if "user_id" not in session:
            return jsonify(error="No autenticado"), 401
        return view(*args, **kwargs)
    return wrapper
```

- **start_session** anota la fecha del ingreso y guarda el id del usuario en la sesión. Es lo mismo para correo y para Google.
- **user_to_dict** decide qué datos del usuario salen en JSON. Fíjate en que el hash de la clave nunca sale.
- **login_required** es un decorador propio: protege cualquier ruta de la API; sin sesión responde 401.

Bloque 4: registro y login

backend/app.py (4 de 6)

```
# ------------------------------------------------------------- API
@app.post("/api/register")
def register():
    data = request.get_json(silent=True) or {}
    name = (data.get("name") or "").strip()
    email = (data.get("email") or "").strip().lower()
    password = data.get("password") or ""

    if not EMAIL_RE.match(email):
        return jsonify(error="Email no válido"), 400
    if len(password) < 6:
        return jsonify(error="La contraseña debe tener al menos 6 caracteres"), 400

    db = get_db()
    try:
        cur = db.execute(
            "INSERT INTO users (name, email, password_hash, created_at) VALUES (?, ?, ?, ?)",
            (name or None, email, generate_password_hash(password), now()),
        )
        db.commit()
    except sqlite3.IntegrityError:
        return jsonify(error="Ese email ya está registrado"), 409

    row = start_session(db, cur.lastrowid)
    return jsonify(message="Usuario registrado", user=user_to_dict(row)), 201


@app.post("/api/login")
def login():
    data = request.get_json(silent=True) or {}
    email = (data.get("email") or "").strip().lower()
    password = data.get("password") or ""

    db = get_db()
    row = db.execute("SELECT * FROM users WHERE email = ?", (email,)).fetchone()
    if row is not None and not row["password_hash"]:
        return jsonify(error="Esta cuenta se creó con Google. Usa 'Ingresar con Google'"), 401
    if row is None or not check_password_hash(row["password_hash"], password):
        return jsonify(error="Email o contraseña incorrectos"), 401

    row = start_session(db, row["id"])
    return jsonify(message="Inicio de sesión exitoso", user=user_to_dict(row))
```

- El backend valida: formato del correo con una expresión regular y clave de al menos 6 caracteres. Aunque la pantalla también lo revise, la regla vive aquí.
- **generate_password_hash** guarda una huella de la clave, no la clave. **check_password_hash** compara al entrar.
- Los valores van siempre con **?**: así un correo como `' OR 1=1 --` es solo texto y no SQL (inyección SQL).
- Códigos: 201 creado, 400 datos mal, 401 no autorizado, 409 ya existe. app.js los usa para decidir qué mostrar.

Bloque 5: Google

backend/app.py (5 de 6)

```
@app.post("/api/google")
def google_login():
    if not GOOGLE_CLIENT_ID:
        return jsonify(error="Google login no configurado: falta GOOGLE_CLIENT_ID en backend/config.py"), 500
    credential = (request.get_json(silent=True) or {}).get("credential")
    if not credential:
```

```
        return jsonify(error="Falta el token de Google"), 400

    try:
        from google.auth.transport import requests as google_requests
        from google.oauth2 import id_token
        info = id_token.verify_oauth2_token(credential, google_requests.Request(), GOOGLE_CLIENT_ID)
    except ImportError:
        return jsonify(error="Instala dependencias: pip install -r requirements.txt"), 500
    except ValueError:
        return jsonify(error="Token de Google inválido"), 401

    if not info.get("email_verified"):
        return jsonify(error="El email de Google no está verificado"), 401

    google_id = info["sub"]
    email = info["email"].lower()
    name = info.get("name")

    db = get_db()
    row = db.execute("SELECT * FROM users WHERE google_id = ? OR email = ?", (google_id, email)).fetchone()
    if row is None:
        cur = db.execute(
            "INSERT INTO users (name, email, password_hash, created_at, google_id) VALUES (?, ?, '', ?, ?)",
            (name, email, now(), google_id),
        )
        user_id = cur.lastrowid
    else:
        user_id = row["id"]
        # vincula la cuenta existente (registrada con email) a Google
        db.execute("UPDATE users SET google_id = ?, name = COALESCE(name, ?) WHERE id = ?",
                   (google_id, name, user_id))
    row = start_session(db, user_id)
    return jsonify(message="Inicio de sesión con Google exitoso", user=user_to_dict(row))
```

- El navegador envía el **credential** (el ID token que le dio Google). **verify_oauth2_token** lo comprueba con Google: que sea auténtico, que no haya vencido y que sea para tu Client ID.
- Solo se aceptan correos verificados por Google. **sub** es el id único de la persona en Google.
- Si no existe el usuario, se crea con clave vacía. Si ya existe con ese correo, se **vincula** (UPDATE con el google_id). Por eso quien se registró con correo y luego entra con Google tiene una sola cuenta.

Bloque 6: el resto de la API y las páginas

backend/app.py (6 de 6)

```
@app.get("/api/config")
def public_config():
    return jsonify(google_client_id=GOOGLE_CLIENT_ID)


@app.get("/api/health")
def health():
    return jsonify(status="ok")


@app.post("/api/logout")
def logout():
    session.clear()
    return jsonify(message="Sesión cerrada")


@app.get("/api/me")
@login_required
def me():
    row = get_db().execute("SELECT * FROM users WHERE id = ?", (session["user_id"],)).fetchone()
    if row is None:
```

```
        session.clear()
        return jsonify(error="No autenticado"), 401
    return jsonify(user=user_to_dict(row))


# ------------------------------------------------------------------ frontend
@app.get("/")
def index():
    return send_from_directory(FRONTEND_DIR, "index.html")


@app.get("/home")
def home():
    # Solo usuarios con sesión pueden ver el home
    if "user_id" not in session:
        return redirect("/")
    return send_from_directory(FRONTEND_DIR, "home.html")


@app.get("/<path:filename>")
def static_files(filename):
    # Lista blanca: solo archivos de primer nivel con extensión permitida.
    # (Antes "/Backend/database.db" o "/./backend/database.db" descargaban la base de datos.)
    if "/" in filename or "\\" in filename or os.path.splitext(filename)[1].lower() not in STATIC_EXTENSIONS:
        return jsonify(error="No encontrado"), 404
    if filename == "home.html":
        return redirect("/home")
    return send_from_directory(FRONTEND_DIR, filename)


init_db()

if __name__ == "__main__":
    print("\n  Abre en el navegador: http://localhost:5000  (celular: http://IP-del-computador:5000)\n")
    # 0.0.0.0: también atiende al emulador (10.0.2.2) y al celular en la misma red Wi-Fi
    app.run(host="0.0.0.0", port=5000, debug=True)
```

- **/api/config** entrega el Client ID a la página para dibujar el botón de Google. **/api/health** solo responde «ok»: sirve para saber si el servidor está vivo.
- **/api/logout** borra la sesión. **/api/me** devuelve el usuario de la sesión; home.html lo usa para mostrar el nombre.
- **/home** es una página: sin sesión redirige al login en vez de responder 401. **/<path:filename>** sirve app.js, mobile.js y las imágenes, solo si pasan la lista blanca.
- **host="0.0.0.0"**: el servidor escucha en toda la red para que entren el emulador y el celular.

1. Guarda y ejecuta: en la terminal, **cd backend** y **python app.py**.

> **Lo que debes ver:** «Running on http://127.0.0.1:5000» y otra línea con la IP de tu computador. Todavía no abras el navegador: falta el frontend. Si hay un error de importación de mobile, es normal: lo creas en el paso 5. Mientras tanto puedes comentar las dos líneas de mobile en app.py.

Paso 4: El frontend (8 min)

1. Reemplaza **index.html** por esta versión. Es tu login de la sesión 9 con ids en los campos, un campo de nombre oculto para el registro, el botón de Google y un párrafo para los mensajes:

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
   <title>Iniciar sesión</title>
</head>
<body>
   <section class="bg-gray-50 min-h-screen flex items-center justify-center">
 <!-- login container -->
 <div class="bg-gray-100 flex rounded-2xl shadow-lg max-w-3xl p-5 items-center">
   <!-- form -->
   <div class="md:w-1/2 px-8 md:px-16">
     <h2 id="form-title" class="font-bold text-2xl text-[#002D74]">Iniciar sesión</h2>
     <p class="text-xs mt-4 text-[#002D74]">Si ya tienes cuenta, entra con tu correo.</p>

     <form id="auth-form" class="flex flex-col gap-4 mt-8">
       <input id="name" class="p-2 rounded-xl border hidden" type="text" name="name"
              placeholder="Nombre" >
       <input class="p-2 rounded-xl border" type="email" name="email"
              placeholder="Correo electrónico" required>
       <div class="relative">
         <input id="password" class="p-2 rounded-xl border w-full" type="password" name="password"
                placeholder="Contraseña" required>
         <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="gray"
              id="toggle-password"
              class="bi bi-eye cursor-pointer absolute top-1/2 right-3 -translate-y-1/2"
              viewBox="0 0 16 16" >
           <path d="M16 8s-3-5.5-8-5.5S0 8 0 8s3 5.5 8 5.5S16 8 16 8zM1.173 8
                 a13.133 13.133 0 0 1 1.66-2.043C4.12 4.668 5.88 3.5 8 3.5
                 c2.12 0 3.879 1.168 5.168 2.457A13.133 13.133 0 0 1 14.828 8
                 c-.058.087-.122.183-.195.288-.335.48-.83 1.12-1.465 1.755
                 C11.879 11.332 10.119 12.5 8 12.5c-2.12 0-3.879-1.168-5.168-2.457
                 A13.134 13.134 0 0 1 1.172 8z" />
           <path d="M8 5.5a2.5 2.5 0 1 0 0 5 2.5 2.5 0 0 0 0-5zM4.5 8
                 a3.5 3.5 0 1 1 7 0 3.5 3.5 0 0 1-7 0z" />
         </svg>
       </div>
<button id="submit-btn" type="submit"
       class="bg-violet-500 hover:bg-violet-600 focus:outline-2 focus:outline-offset-2
              focus:outline-violet-500 active:bg-violet-700 text-white py-2 rounded-xl"
       >Ingresar</button>
       <p id="message" class="text-sm"></p>
     </form>

     <div class="mt-6 grid grid-cols-3 items-center text-gray-400">
       <hr class="border-gray-400">
       <p class="text-center text-sm">o</p>
       <hr class="border-gray-400">
     </div>

     <div id="google-btn" class="mt-5 flex justify-center hidden"></div>
     <button id="google-fallback" type="button"
            class="bg-white border py-2 w-full rounded-xl mt-5 flex justify-center items-center
                   text-sm hover:scale-105 duration-300 text-[#002D74]"
            >
       <svg class="mr-3" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 48 48" width="25px">
         <path fill="#FFC107" d="M43.611,20.083H42V20H24v8h11.303c-1.649,4.657-6.08,8-11.303,8
               c-6.627,0-12-5.373-12-12c0-6.627,5.373-12,12-12
```

```
                c3.059,0,5.842,1.154,7.961,3.039l5.657-5.657C34.046,6.053,29.268,4,24,4
                C12.955,4,4,12.955,4,24c0,11.045,8.955,20,20,20c11.045,0,20-8.955,20-20
                C44,22.659,43.862,21.35,43.611,20.083z" />
          <path fill="#FF3D00" d="M6.306,14.691l6.571,4.819C14.655,15.108,18.961,12,24,12
                c3.059,0,5.842,1.154,7.961,3.039l5.657-5.657C34.046,6.053,29.268,4,24,4
                C16.318,4,9.656,8.337,6.306,14.691z" />
          <path fill="#4CAF50" d="M24,44c5.166,0,9.86-1.977,13.409-5.192l-6.19-5.238
                C29.211,35.091,26.715,36,24,36c-5.202,0-9.619-3.317-11.283-7.946
                l-6.522,5.025C9.505,39.556,16.227,44,24,44z" />
          <path fill="#1976D2" d="M43.611,20.083H42V20H24v8h11.303c-0.792,2.237-2.231,4.166-4.087,5.571
                c0.001-0.001,0.002-0.001,0.003-0.002l6.19,5.238C36.971,39.205,44,34,44,24
                C44,22.659,43.862,21.35,43.611,20.083z" />
        </svg>
        Ingresar con Google
      </button>

      <div class="mt-5 text-xs border-b border-[#002D74] py-4 text-[#002D74]">
        <a href="#">¿Olvidaste tu contraseña?</a>
      </div>

      <div class="mt-3 text-xs flex justify-between items-center text-[#002D74]">
        <p id="toggle-text">¿No tienes cuenta?</p>
        <button id="toggle-mode" type="button"
                class="py-2 px-5 bg-white border rounded-xl hover:scale-110 duration-300"
                >Regístrate</button>
      </div>
    </div>

    <!-- image -->
    <div class="md:block hidden w-1/2">
      <img class="rounded-2xl" src="login.jpg" alt="Imagen de bienvenida">
    </div>
  </div>
</section>
<script src="app.js"></script>



</body>
 </html>
```

- **id="name"** con la clase **hidden**: solo aparece cuando el formulario está en modo registro.
- **#google-btn** es donde Google dibuja su botón; **#google-fallback** es el botón de respaldo que avisa si Google no está configurado.
- **#toggle-mode** cambia entre Iniciar sesión y Crear cuenta sin cambiar de página.
- Al final, **app.js**: todo el comportamiento está ahí.

2. Crea **app.js** en la carpeta login_screen:

app.js

```
// Conecta el formulario de login con el backend Flask (/api/*)
// La página se sirve desde Flask: la dirección es la misma desde la que se abrió
// (localhost en el computador, 10.0.2.2 en el emulador, la IP del computador en el celular).
const SERVER = location.origin;

const form = document.getElementById("auth-form");
const title = document.getElementById("form-title");
const nameInput = document.getElementById("name");
const submitBtn = document.getElementById("submit-btn");
const toggleBtn = document.getElementById("toggle-mode");
const toggleText = document.getElementById("toggle-text");
const msg = document.getElementById("message");
const pwd = document.getElementById("password");
const eye = document.getElementById("toggle-password");
```

```
const googleBtn = document.getElementById("google-btn");
const googleFallback = document.getElementById("google-fallback");

let mode = "login";

function showMessage(text, ok) {
  msg.textContent = text;
  msg.className = "text-sm " + (ok ? "text-green-600" : "text-red-600");
}

const SERVER_DOWN =
  "El servidor no está corriendo. Abre una terminal en la carpeta backend y ejecuta: python app.py";

// 1) La página debe abrirse desde el servidor Flask (http://localhost:5000).
//    Si se abrió con doble clic (file://) o con Live Server, redirigimos allí.
if (location.protocol === "file:") {
  fetch("http://localhost:5000/api/health", { mode: "no-cors" })
    .then(() => { location.href = "http://localhost:5000/"; })
    .catch(() => showMessage(SERVER_DOWN, false));
} else {
  init();
}

function init() {
  // Si ya hay sesión activa, ir directo al home
  fetch("/api/me").then(r => { if (r.ok) location.href = "/home"; }).catch(() => {});
  setupGoogle();
}

function setMode(newMode) {
  mode = newMode;
  const reg = mode === "register";
  title.textContent = reg ? "Crear cuenta" : "Iniciar sesión";
  submitBtn.textContent = reg ? "Crear cuenta" : "Ingresar";
  nameInput.classList.toggle("hidden", !reg);
  toggleText.textContent = reg ? "¿Ya tienes cuenta?" : "¿No tienes cuenta?";
  toggleBtn.textContent = reg ? "Iniciar sesión" : "Regístrate";
  msg.textContent = "";
}

toggleBtn.addEventListener("click", () => setMode(mode === "login" ? "register" : "login"));
eye.addEventListener("click", () => { pwd.type = pwd.type === "password" ? "text" : "password"; });

async function postJSON(url, body) {
  let res;
  try {
    res = await fetch(url, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });
  } catch {
    throw new Error(SERVER_DOWN);
  }
  let data;
  try { data = await res.json(); } catch {
    throw new Error(`Respuesta inesperada del servidor (${res.status}). Reinicia python app.py`);
  }
  if (!res.ok) throw new Error(data.error || "Error");
  return data;
}

form.addEventListener("submit", async (e) => {
  e.preventDefault();
  try {
    const data = await postJSON(`/api/${mode}`, {
      email: form.email.value,
      password: form.password.value,
      name: nameInput.value,
```

```
    });
    showMessage(`${data.message}. Redirigiendo...`, true);
    location.href = "/home";
  } catch (err) {
    showMessage(err.message, false);
  }
});

// ---------------------------------------------------------------- Google
async function setupGoogle() {
  let clientId = "";
  try {
    clientId = (await (await fetch("/api/config")).json()).google_client_id;
  } catch {}

  if (!clientId) {
    googleFallback.addEventListener("click", () =>
      showMessage("Login con Google no configurado: pega tu GOOGLE_CLIENT_ID en backend/config.py y reinicia el servidor.", false)
    return;
  }

  const script = document.createElement("script");
  script.src = "https://accounts.google.com/gsi/client";
  script.async = true;
  script.onload = () => {
    google.accounts.id.initialize({ client_id: clientId, callback: onGoogleCredential });
    google.accounts.id.renderButton(googleBtn, {
      theme: "outline", size: "large", shape: "pill", text: "signin_with", width: googleFallback.offsetWidth || 240,
    });
    googleFallback.classList.add("hidden");
    googleBtn.classList.remove("hidden");
  };
  document.head.appendChild(script);
}

async function onGoogleCredential(response) {
  try {
    const data = await postJSON("/api/google", { credential: response.credential });
    showMessage(`${data.message}. Redirigiendo...`, true);
    location.href = "/home";
  } catch (err) {
    showMessage(err.message, false);
  }
}
```

- **SERVER = location.origin**: la dirección desde la que se abrió la página. Así funciona en localhost, en el emulador y en el celular sin cambiar nada.
- Si la página se abre con doble clic (file://), app.js comprueba si el servidor está vivo y redirige a localhost:5000.
- **init()**: si ya hay sesión, va directo a /home. Luego prepara Google.
- **setMode** cambia los textos y muestra u oculta el campo de nombre: un solo formulario para dos usos.
- **postJSON** hace el fetch, lee el JSON y, si el código no es 2xx, lanza un error con el mensaje del servidor; por eso el catch muestra «Email o contraseña incorrectos» sin más código.
- **setupGoogle** pide el Client ID a /api/config; si está, carga la librería de Google y dibuja el botón; al elegir cuenta, **onGoogleCredential** envía el token a /api/google.

3. Crea **home.html**:

home.html

```
<!DOCTYPE html>
<html lang="es">
<head>
```

```
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
  <title>Inicio</title>
</head>
<body class="bg-gray-50 min-h-screen">
  <!-- navbar -->
  <nav class="bg-white shadow">
    <div class="max-w-5xl mx-auto px-6 py-4 flex justify-between items-center">
      <span class="font-bold text-xl text-[#002D74]">MiApp</span>
      <div class="flex items-center gap-4">
        <span id="nav-user" class="text-sm text-gray-600"></span>
        <button id="logout-btn" class="bg-violet-500 hover:bg-violet-600 text-white text-sm py-2 px-4 rounded-xl">Cerrar sesión</
      </div>
    </div>
  </nav>

  <!-- contenido -->
  <main class="max-w-5xl mx-auto px-6 py-10">
    <div class="bg-gray-100 rounded-2xl shadow-lg p-8 flex flex-col md:flex-row items-center gap-8">
      <div class="md:w-1/2">
        <h1 class="font-bold text-3xl text-[#002D74]">Hola, <span id="user-name"></span></h1>
        <p class="mt-4 text-[#002D74]">Iniciaste sesión correctamente. Esta página solo se ve con sesión activa.</p>
        <ul class="mt-6 text-sm text-gray-600 space-y-1">
          <li><b>Correo:</b> <span id="user-email"></span></li>
          <li><b>Cuenta creada:</b> <span id="user-created"></span></li>
          <li><b>Último ingreso:</b> <span id="user-last"></span></li>
        </ul>
      </div>
      <div class="md:w-1/2">
        <img class="rounded-2xl" src="login.jpg" alt="Imagen de bienvenida">
      </div>
    </div>

    <!-- base de conocimiento de apps móviles (mobile.js) -->
    <section class="mt-10 bg-gray-100 rounded-2xl shadow-lg p-6 md:p-8">
      <div class="flex flex-col md:flex-row md:items-end md:justify-between gap-4">
        <div>
          <h2 class="font-bold text-2xl text-[#002D74]">Apps móviles</h2>
          <p id="mobile-stats" class="text-sm text-gray-600 mt-1">Base de conocimiento interna</p>
        </div>
        <input id="mobile-search" type="search" placeholder="Buscar (ej. flutter, seguridad, push)..."
               class="p-2 rounded-xl border bg-white w-full md:w-72">
      </div>
      <div id="mobile-tabs" class="flex flex-wrap gap-2 mt-6"></div>
      <div id="mobile-chips" class="flex flex-wrap gap-2 mt-3"></div>
      <div id="mobile-list" class="grid grid-cols-1 md:grid-cols-2 gap-4 mt-6"></div>
    </section>
  </main>

  <script>
    // Debe abrirse desde el servidor Flask
    if (location.protocol === "file:") location.href = "http://localhost:5000/home";

    const fmt = (s) => s ? new Date(s).toLocaleString() : "—";

    fetch("/api/me")
      .then(r => { if (!r.ok) throw 0; return r.json(); })
      .then(({ user }) => {
        const name = user.name || user.email;
        document.getElementById("user-name").textContent = name;
        document.getElementById("nav-user").textContent = user.email;
        document.getElementById("user-email").textContent = user.email;
        document.getElementById("user-created").textContent = fmt(user.created_at);
        document.getElementById("user-last").textContent = fmt(user.last_login);
      })
      .catch(() => { window.location.href = "/"; });

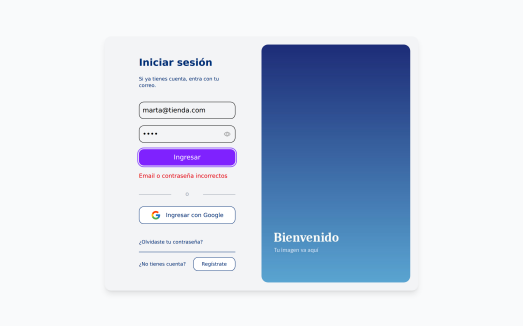
    document.getElementById("logout-btn").addEventListener("click", async () => {
```

```
      await fetch("/api/logout", { method: "POST" });
      window.location.href = "/";
    });
  </script>
  <script src="mobile.js"></script>
</body>
</html>
```

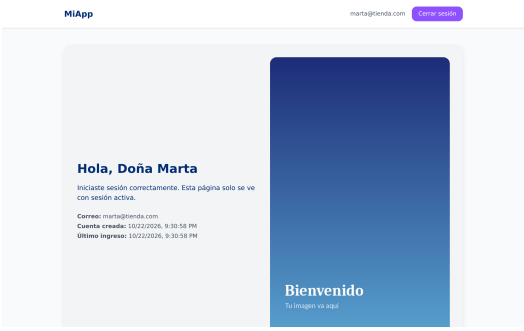
- Pide **/api/me**; con la respuesta rellena nombre, correo y fechas. Si responde 401, vuelve al login.
- El botón Cerrar sesión hace POST a /api/logout y vuelve al login.
- La sección «Apps móviles» la llena mobile.js en el paso 5.

4. Reinicia el servidor (Ctrl+C y python app.py) y abre **http://localhost:5000**.

> **Lo que debes ver:** tu login. Toca Regístrate, crea una cuenta y entra: el inicio con tu nombre. Cierra sesión y entra con una clave equivocada: el mensaje en rojo, sin recargar.



Clave equivocada: 401 y mensaje en rojo



Sesión iniciada: /home con los datos de /api/me

Paso 5: La API de conocimiento (5 min)

Esta parte es un ejemplo de API de datos: tablas que se llenan desde listas de Python, rutas con filtros y sesión obligatoria. Es el modelo del backend que tendrá la app de tu proyecto. Los dos archivos grandes se copian del ZIP del aula virtual: **backend/mobile_data.py** (el contenido: plataformas, frameworks, artículos y glosario) y **mobile.js** (las tarjetas del home).

1. Copia **backend/mobile_data.py** y **mobile.js** del ZIP a sus carpetas.
2. Crea **backend/mobile.py**. Mira la primera parte: así se define un blueprint y se exige sesión a todas sus rutas.

backend/mobile.py (inicio; el archivo completo está en el ZIP)

```
"""
API interna de conocimiento sobre aplicaciones móviles (/api/mobile/*).

Todas las rutas requieren sesión iniciada. El contenido vive en mobile_data.py
y se vuelca a SQLite al arrancar con seed_mobile_data().
"""
import json
import unicodedata

from flask import Blueprint, jsonify, request, session

import mobile_data
from db import get_db

bp = Blueprint("mobile", __name__, url_prefix="/api/mobile")

# columnas que se guardan como JSON (listas)
JSON_FIELDS = {"languages", "package_formats", "key_points", "platforms", "pros", "cons", "tags"}


# ---------------------------------------------------------------- esquema y datos
def init_mobile_tables(conn):
    conn.executescript(
        """
        CREATE TABLE IF NOT EXISTS mobile_platforms (
            slug TEXT PRIMARY KEY, name TEXT NOT NULL, vendor TEXT, languages TEXT, ide TEXT,
            store TEXT, package_formats TEXT, developer_fee TEXT, market_share TEXT,
            summary TEXT, key_points TEXT
        );
        CREATE TABLE IF NOT EXISTS mobile_frameworks (
            slug TEXT PRIMARY KEY, name TEXT NOT NULL, type TEXT, company TEXT, language TEXT,
            platforms TEXT, first_release TEXT, summary TEXT, pros TEXT, cons TEXT, use_when TEXT
        );
        CREATE TABLE IF NOT EXISTS mobile_articles (
            slug TEXT PRIMARY KEY, title TEXT NOT NULL, category TEXT, level TEXT, tags TEXT,
            summary TEXT, content TEXT
        );
        CREATE TABLE IF NOT EXISTS mobile_glossary (
            slug TEXT PRIMARY KEY, term TEXT NOT NULL, definition TEXT NOT NULL
        );
        """
    )


def _replace_all(conn, table, rows):
    """Sustituye el contenido de la tabla por `rows` (lista de dicts)."""
    conn.execute(f"DELETE FROM {table}")
    for row in rows:
        values = {k: json.dumps(v, ensure_ascii=False) if k in JSON_FIELDS else v for k, v in row.items()}
        cols = ", ".join(values)
        marks = ", ".join("?" for _ in values)
        conn.execute(f"INSERT INTO {table} ({cols}) VALUES ({marks})", tuple(values.values()))
def seed_mobile_data(conn):
```

```
    init_mobile_tables(conn)
    _replace_all(conn, "mobile_platforms", mobile_data.PLATFORMS)
    _replace_all(conn, "mobile_frameworks", mobile_data.FRAMEWORKS)
    _replace_all(conn, "mobile_articles", mobile_data.ARTICLES)
    _replace_all(conn, "mobile_glossary",
                 [{"slug": s, "term": t, "definition": d} for s, t, d in mobile_data.GLOSSARY])

...

@bp.before_request
def require_login():
    if "user_id" not in session:
        return jsonify(error="No autenticado"), 401

...

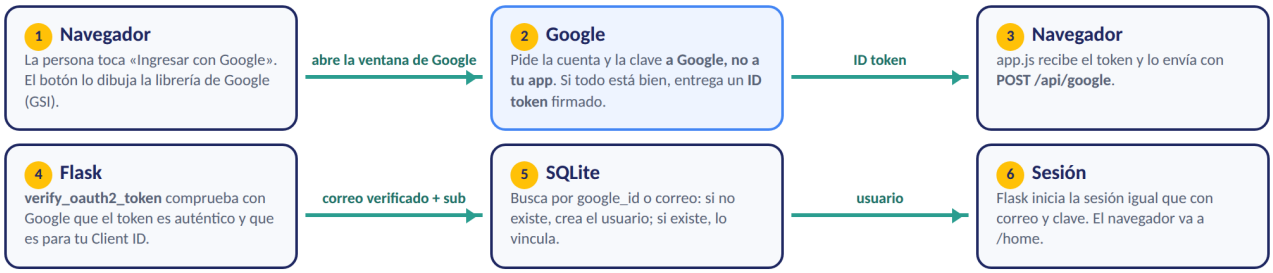
@bp.get("/frameworks")
def frameworks():
    """Filtros opcionales: ?type=nativo|multiplataforma|híbrido &platform=iOS &q=texto"""
    items = _fetch("mobile_frameworks")
    if fw_type := request.args.get("type"):
        items = [i for i in items if _normalize(i["type"]) == _normalize(fw_type)]
    if platform := request.args.get("platform"):
        items = [i for i in items if _normalize(platform) in map(_normalize, i["platforms"])]
    return jsonify(items=_filter_by_query(items))
```

- **Blueprint("mobile", url_prefix="/api/mobile")**: todas las rutas del archivo cuelgan de /api/mobile. **before_request** revisa la sesión antes de cualquiera.
- **seed_mobile_data** vuelca las listas de mobile_data.py a las tablas al arrancar: para cambiar el contenido se edita la lista y se reinicia.
- Los filtros viajan en la URL (**?type=nativo**, **?q=flutter**): el backend filtra, el frontend solo pinta.

3. Reinicia el servidor, entra y mira el home: la sección «Apps móviles» con pestañas y buscador.
4. Con la sesión iniciada, abre en el navegador **http://localhost:5000/api/mobile/search?q=flutter**.

> **Lo que debes ver:** el JSON crudo de la respuesta: así se ve una API por dentro. Cierra sesión y vuelve a abrir la misma dirección: {"error": "No autenticado"}.

Paso 6: Login con Google (7 min)



Lo que pasa al tocar «Ingresar con Google»: la persona se identifica ante Google; tu backend solo verifica el token.

1. Entra a **console.cloud.google.com** con tu cuenta de Google. Arriba a la izquierda, crea un proyecto (por ejemplo «login-screen») o elige uno.

2. Menú → **APIs y servicios → Pantalla de consentimiento de OAuth**. Tipo de usuario **Externo**, nombre de la app y tu correo de contacto. Guarda; el resto puede quedar por defecto.
3. Menú → **APIs y servicios → Credenciales → Crear credenciales → ID de cliente de OAuth**. Tipo de aplicación: **Aplicación web**.
4. En **Orígenes autorizados de JavaScript** agrega dos: **http://localhost** y **http://localhost:5000**. No hace falta URI de redirección. Crear.
5. Copia el **ID de cliente** (termina en .apps.googleusercontent.com) y pégalo en **backend/config.py**:

backend/config.py

```
GOOGLE_CLIENT_ID = "1234567890-abcdefg.apps.googleusercontent.com"
```

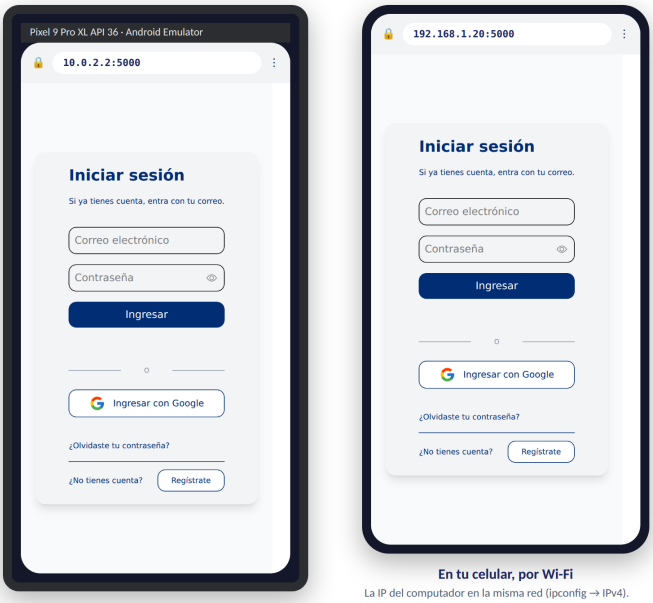
6. Reinicia el servidor y recarga el login.

> **Lo que debes ver:** el botón azul de Google en lugar del de respaldo. Al tocarlo, la ventana de Google con tus cuentas; al elegir una, entras al inicio con el nombre de tu cuenta de Google.

7. Comprueba las reglas: cierra sesión e intenta entrar con ese mismo correo y una clave: «Esta cuenta se creó con Google». Y si te habías registrado antes con correo, al entrar con Google sigues siendo el mismo usuario (mira el id en /api/me).

Límites de Google. El botón solo funciona en los orígenes que registraste: en localhost. Desde el emulador (10.0.2.2) o desde el celular por IP, Google lo rechaza; es normal y no es un error tuyo. Para que funcione en un celular, la app tendría que estar publicada en un dominio real con HTTPS. Si en el salón no se puede entrar a la consola de Google, este paso queda para la casa: todo lo demás del taller funciona sin él.

Paso 7: Probar en el emulador y en el celular (10 min)



En el emulador de Android Studio
10.0.2.2 es «el computador» visto desde el emulador.

En tu celular, por Wi-Fi
La IP del computador en la misma red (ipconfig → IPv4).

El mismo servidor, visto desde el emulador y desde un celular real.

En el emulador de Android Studio

1. Con app.py corriendo, abre Android Studio y arranca el emulador (Device Manager →).
2. En el emulador abre Chrome y escribe **http://10.0.2.2:5000**. Inicia sesión con tu cuenta de correo y toma la captura del inicio.

En tu celular, por Wi-Fi

1. En la terminal del computador, **ipconfig**: busca **Dirección IPv4** del adaptador Wi-Fi (192.168.x.x). Flask también la muestra al arrancar.
2. Conecta el celular a la **misma Wi-Fi** y abre **http://[esa IP]:5000**. Si Windows pregunta por el firewall, elige Permitir.
3. Entra, revisa que todo se toque bien con el dedo y toma la captura del inicio.
4. En Chrome del computador: **F12 → Network**, inicia sesión y haz clic en la fila **login**: en Payload lo que enviaste, en Response lo que respondió. Captura.

> **Si el celular no llega:** muchas redes de universidad aíslan los dispositivos. Plan B: activa el hotspot del celular, conecta el computador a esa red, vuelve a mirar ipconfig y entra a la nueva dirección. Si sigue sin llegar, el firewall: Configuración → Firewall → Permitir una aplicación → Python, red privada.

Evidencia

- La carpeta login_screen comprimida, sin .venv y sin database.db.

- Captura del inicio (/home) en el emulador y en tu celular.
- Captura de la pestaña Network con el POST a /api/login y su respuesta.
- Captura del botón de Google funcionando o, si no fue posible, del mensaje de «no configurado».
- En la bitácora, con tus palabras: el viaje del dato en el login, y qué pasa distinto cuando se entra con Google.
- Se sube al aula virtual antes del [dd/mm/2026].

Problemas comunes

Lo que ves	Causa probable	Solución
«python no se reconoce como un comando»	Python no quedó en el PATH	Reinstala marcando Add python.exe to PATH y abre una terminal nueva.
«la ejecución de scripts está deshabilitada»	Política de PowerShell	Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser.
«No module named flask» (o google)	Instalaste fuera del entorno, o no está activo	Revisa que la línea empiece con (.venv) y repite pip install -r backend\requirements.txt.
«No module named mobile»	Falta backend/mobile.py o mobile_data.py	Cópialos del ZIP (paso 5) o comenta las dos líneas de mobile en app.py mientras tanto.
La página sale sin estilos	No cargó Tailwind desde el CDN	Revisa internet o cambia de red.
Al tocar Ingresar no pasa nada o se recarga	app.js no cargó, o los ids no coinciden	F12 → Console muestra el error. Revisa que app.js esté junto a index.html y los ids del paso 4.
«El servidor no está corriendo»	Abriste index.html con doble clic y app.py no corre	Arranca python app.py y entra por http://localhost:5000.
El botón de Google dice «no configurado»	config.py sin Client ID o servidor sin reiniciar	Pega el ID en config.py y reinicia python app.py.
Google: «origin not allowed» o el botón no aparece	El origen no está en la lista de Google	Agrega http://localhost y http://localhost:5000 en Credenciales y espera unos minutos. Entra siempre por localhost, no por 127.0.0.1.
El emulador no abre la página	Usaste localhost en vez de 10.0.2.2, o el servidor no corre	En el emulador siempre http://10.0.2.2:5000.
El celular no llega al servidor	Redes distintas, red que aísla, firewall, o host="localhost"	Misma Wi-Fi o hotspot; permitir Python en el firewall; app.run(host="0.0.0.0").